

Topological Sort:

Date ____/____/____
Page ____

* Topological Sort

```
void topological sort (int v, int adj[v][v])
{
    int indegree[v], queue[v];
    int front = 0, rear = -1;

    for (int i = 0; i < v; i++)
        indegree[i] = 0;
    for (int i = 0; i < v; i++)
    {
        for (int j = 0; j < v; j++)
        {
            if (adj[i][j] == 1)
                indegree[j]++;
        }
    }

    for (int i = 0; i < v; i++)
        if (indegree[i] == 0)
            queue[++rear] = i;

    printf("\n Topological order");

    while (front <= rear)
    {
        int node = queue[front++];
        printf(" %d", node);
        for (int i = 0; i < v; i++)
        {
            if (adj[node][i] == 1)
            {
                indegree[i]--;
                if (indegree[i] == 0)
                    queue[++rear] = i;
            }
        }
    }
}
```

```

        queue[++rear] = i;
    }
}

}

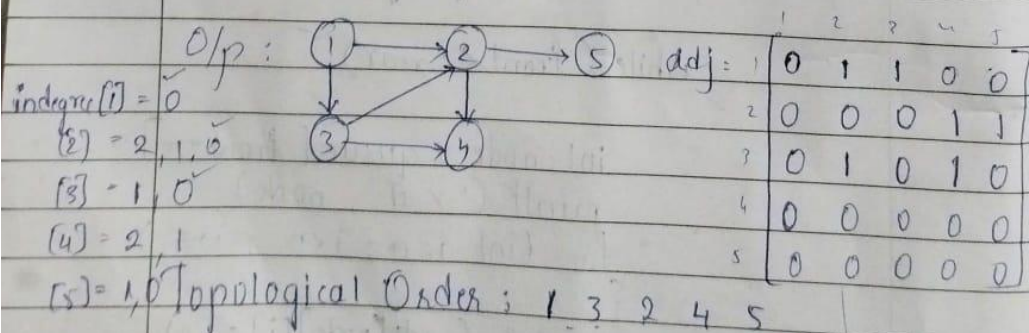
}

int main()
{
    int v, adj[10][10];
    printf("Enter the number of vertices:");
    scanf("%d", &v);

    printf("Enter the adjacency matrix:");
    for (int i = 0; i < v; i++)
        for (int j = 0; j < v; j++)
            scanf("%d", &adj[i][j]);

    topologicalSort(v, adj);
    return 0;
}

```



Johnson Trotter:

Date 9/3/26
Page _____

Program :

```
#include <stdio.h>

int LR=1; RL=0;

void swap(int *a, int *b)
{
    int temp=*a;
    *a=*b;
    *b=temp;
}

int fact(int n)
{
    int result=1;
    for (int i=1; i<=n; i++)
        result=result*i;
    return result;
}

int permutation (int a[], int dir[], int n)
{
    int prev=0; mobile=0; pos;
    for (int i=0; i<n; i++)
    {
        if (dir[a[i]-1] == RL && i!=0)
        {
            if (a[i]>a[i-1] && a[i]>prev)
            {
                mobile=a[i];
                prev=mobile;
            }
        }
        if (dir[a[i]-1] == LR && i==n-1)
```



```

    {
        if (a[i] > a[i+1] && a[i] > prev)
        {
            mobile = a[i];
            prev = mobile;
        }
    }

    for (int i=0; i<n; i++)
    {
        if (a[i] == mobile)
            pos = i+1;
    }

    if (dir[a[pos-1]-1] == RL)
        swap(a[pos-1], a[pos-2]);
    else if (dir[a[pos-1]-1] == LR)
        swap(a[pos], a[pos-1]);

    for (int i=0; i<n; i++)
    {
        if (a[i] > mobile)
        {
            if (dir[a[i]-1] == LR)
                dir[a[i]-1] = RL;
            else if (dir[a[i]-1] == RL)
                dir[a[i]-1] = LR;
        }
    }

    for (int i=0; i<n; i++)
        printf("%d\n", a[i]);
}

```

```
int main()
{
    int n;
    printf("Enter the number of n:");
    scanf("%d", &n);

    int a[n], dir[n];
    for (int i=0; i<n; i++)
    {
        a[i] = i+1;
        printf("%d", a[i]);
    }
    for (int i=0; i<n; i++)
        dir[i] = R2;

    for (int i=1; i<= fact(n); i++)
        permutation(a, dir, n);
    return 0;
}
```

~~Joshwa~~

Merge Sort:

* Merge Sort

Algorithm

1) Start

2) Take 2 sorted arrays as inputs

3) Compare between elements of array 1 & array 2

if array 1 element > array 2

put it in new array

$i++$ & $k++$

else put array 2 element in new array

$j++$ & $k++$

4) Check if first array is empty

if not empty

shift all elements to new array

$i++$ & $k++$

5) Check if array 2 is empty.

if not empty

shift all elements to new array

$j++$ & $k++$

6) Stop.

Program :

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int low, mid, high, n, m;
```

```
int a[50], b[50];
```

```
printf("Enter the size of arrays:");
```

```
scanf("%d %d", &m, &n);
```

```
printf("Enter the array 1 elements:");
```

```
for (int i=0; i<m; i++)
```

```
scanf("%d", &a[i]);
```



```

printf("Enter array 2 elements:");
for (int j=0; j<n; j++)
    scanf("%d", &b[j]);

merge sort (m,n,a,b);
}

void mergesort (int m, int n, int a[], int b[])
{
    int i=0, j=0, c[50];
    while (i<m & j<n)
    {
        if (a[i] < b[j])
        {
            c[k] = a[i];
            i++;
            k++;
        }
        else
        {
            c[k] = b[j];
            j++;
            k++;
        }
    }
    while (i<m)
    {
        c[k] = a[i];
        i++;
        k++;
    }
    while (j<n)
    {
        c[k] = b[j];
        j++;
        k++;
    }
}

```

```
printf("Merged array :");
```

```
for (int i=0; i<(m+n); i++)  
    printf("%d\t", c[i]);  
return 0;
```

```
}
```

O/p: m=10, n=10

Enter array1 elements : 1 3 5 7 9 11 13 15 17 19

Enter array2 elements : 0 2 4 6 8 10 12 14 16 18

Merged array : 0 1 2 3 4 5 6 7 8 9 10 11
12 13 14 15 16 17 18 19

Execution time : 0.00100.

~~0.036666~~

Quick Sort:

Date: / /
Page:

Program:

```
#include <stdio.h>

int QuickSort(int a[], int low, int high)
{
    int j;
    if (low < high)
    {
        j = partition(a, low, high);
        quickSort(a, low, j-1);
        quickSort(a, j+1, high);
    }
}

int partition(int a[], int low, int high)
{
    int i, j, temp, key;
    i = low+1;
    j = high;
    key = a[low];
    while (1)
    {
        while (a[i] <= key & i <= high)
            i++;
        while (a[j] > key)
            j--;
        if (i < j)
        {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
        else
        {
            temp = a[low];
            a[low] = a[j];
            a[j] = temp;
        }
    }
}
```

```
        a[j] = temp;  
        return j;  
    }  
}  
int main()  
{  
    int n;  
    printf("Enter the number of elements:");  
    scanf("%d", &n);  
  
    int a[n];  
    printf("Enter the elements:");  
    for (int i = 0; i < n; i++)  
        scanf("%d", &a[i]);  
    quickSort(a, 0, n-1);  
    printf("Sorted array:");  
    for (int i = 0; i < n; i++)  
        printf("%d", a[i]);  
    return 0;  
}
```

o/p :

Enter the number of elements : 5

Enter the elements :

10

7

8

9

15

Sorted array: 7 8 9 10 15

Heapsort:

Program:

```
#include <stdio.h>
#include <time.h>
```

```
void heapify (int a[], int n, int p)
```

```
{
```

```
    int c, item;
```

```
    item = a[p];
```

```
    c = 2*p + 1;
```

```
    while (c < n)
```

```
    {    if (c + 1 < n)
```

```
        if (a[c] < a[c + 1])
```

```
            c++;
```

```
        if (item < a[c])
```

```
        {
```

```
            a[p] = a[c];
```

```
            p = c;
```

```
            c = 2*p + 1;
```

```
        }
```

```
        else
```

```
            break;
```

```
    } a[p] = item;
```

```
}
```

```
void build_heap (int a[], int n)
```

```
{
```

```
    for (int i = (n - 1) / 2; i >= 0; i++)
```

```
        heapify (a, n, i)
```

```
}
```



```

void heapsort (int a[], int n)
{
    int i, temp;
    build_heap (a, n);

    for (i = n-1; i > 0; i--)
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify (a, i, 0);
    }
    printf ("Sorted array:");
    for (i = 0; i < n; i++)
        printf ("%d\t", a[i]);
}

```

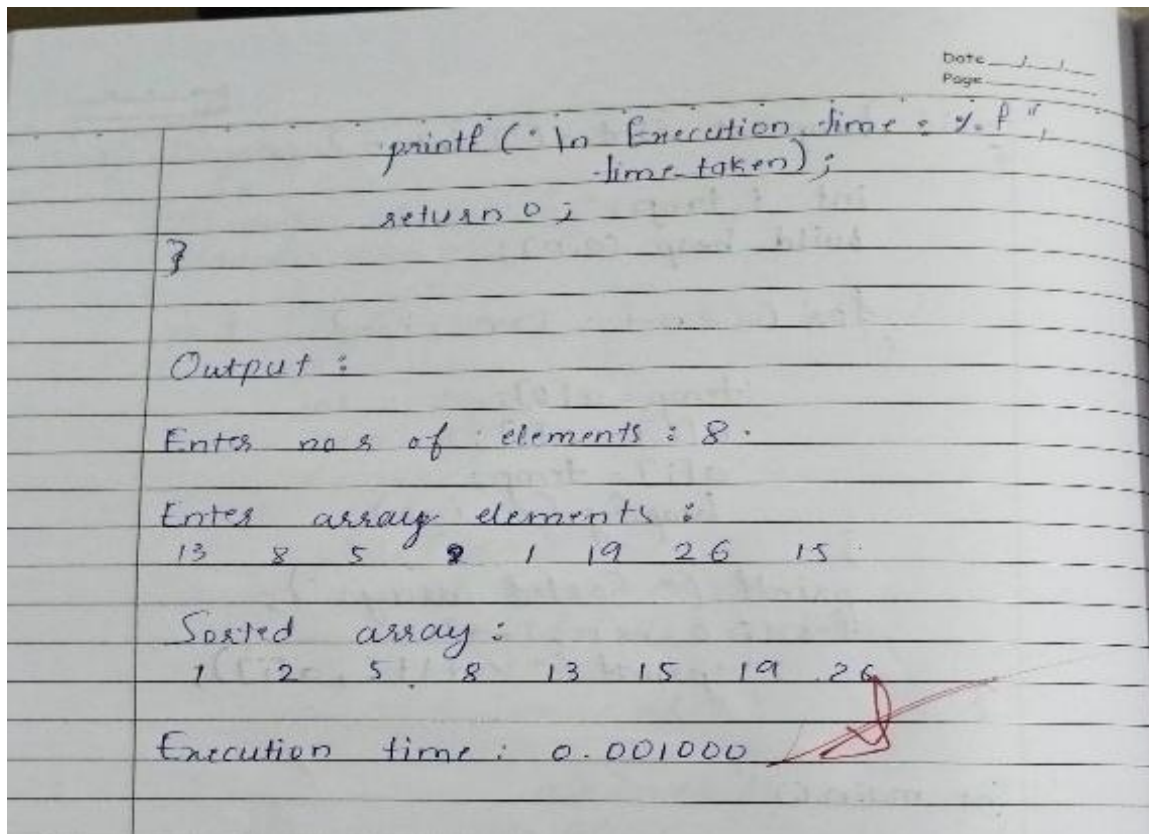
```

int main ()
{
    clock_t start, end;
    int a[100], n, i;

    printf ("Enter number of elements:");
    scanf ("%d", &n);
    printf ("Enter array elements:\n");
    for (i = 0; i < n; i++)
        scanf ("%d", &a[i]);

    start = clock();
    heapsort (a, n);
    end = clock();
    double time_taken = (double)(end-start) /
        CLOCKS_PER_SEC;
}

```



0/1 KnapSack problem:

Date ___/___/___
Page _____

```
Program :
#include <stdio.h>
#include <time.h>
int max (int a, int b)
{
    if (a > b)
        return a;
    return b;
}

int main ()
{
    clock_t start, end;
    int P[100], w[100], M, n, Table[100][100];
    printf ("Enter the number of items: ");
    scanf ("%d", &n);
    printf ("Enter maximum capacity of the
            knapsack: ");
    for (int i = 1; i <= n; i++)
    {
        printf ("Enter the weight & profit
                of item: %d", i);
        scanf ("%d %d", &w[i], &P[i]);
    }

    start = clock ();
    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= M; j++)
            if (i == 0 || j == 0)
            {
                Table[i][j] = 0;
            }
            else if (j < w[i])
            {
                Table[i][j] = Table[i-1][j];
            }
            else
            {
                Table[i][j] = max (Table[i-1][j], P[i] + Table[i-1][j-w[i]]);
            }
    end = clock ();
}
```



```

- dse
{
    Table[i][j] = max (Table[i-1][j],
                      p[i] + Table[i-1][j-w[i]]);
}
}

} end = clock ()
printf ("Maximum profit made : %d",
        Table[n][m]);
double time = (double) (end - start) / CLOCKS_
               PER_SEC;
printf ("Execution time : %.f", time);
}

```

O/p:

Enter number of items : 3

Enter maximum capacity of Knapsack : 4

Enter weight & profit of item 1 : 2 16

Enter weight & profit of item 2 : 2 6

Enter weight & profit of item 3 : 1 18

Maximum profit made : 34

Execution time : 0.000000

Floyd's Algorithm:

Date ___/___/___
Page _____

* Floyd's Algorithm

```
#include <stdio.h>
#include <time.h>
#define INF 99

void floyd (int A[][10], int n)
{
    int i, j, k;
    for (k=0; k<n; k++)
        for (i=0; i<n; i++)
            for (j=0; j<n; j++)
                if (A[i][k] + A[k][j] < A[i][j])
                    A[i][j] = A[i][k] + A[k][j];

    printf("\n Shortest distance matrix :");
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
        {
            if (A[i][j] == INF)
                printf("INF");
            else
                printf("%d", A[i][j]);
        }
    printf("\n");
}

int main ()
{
    int i, j, k, n;
    int cost[10][10], A[10][10];
    printf("Enter num of vertices:");
    scanf ("%d", &n);
    printf("Enter the cost matrix:\n");
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            scanf ("%d", &cost[i][j]);
```

```

        A[i][j] = cost[i][j];
    }
    clock = t_start, end
    start = clock();
    Floyd(A, n);
    end = clock();
    double time = (double)(end - start) /
                  CLOCKS_PER_SEC;
    printf("Execution time: %f", time);
    return 0;

```

o/p :

Enter the number of vertices : 3

Enter the cost matrix :

0	4	99	5	99
99	0	1	99	6
2	99	0	3	99
99	99	1	0	2
1	99	99	4	0

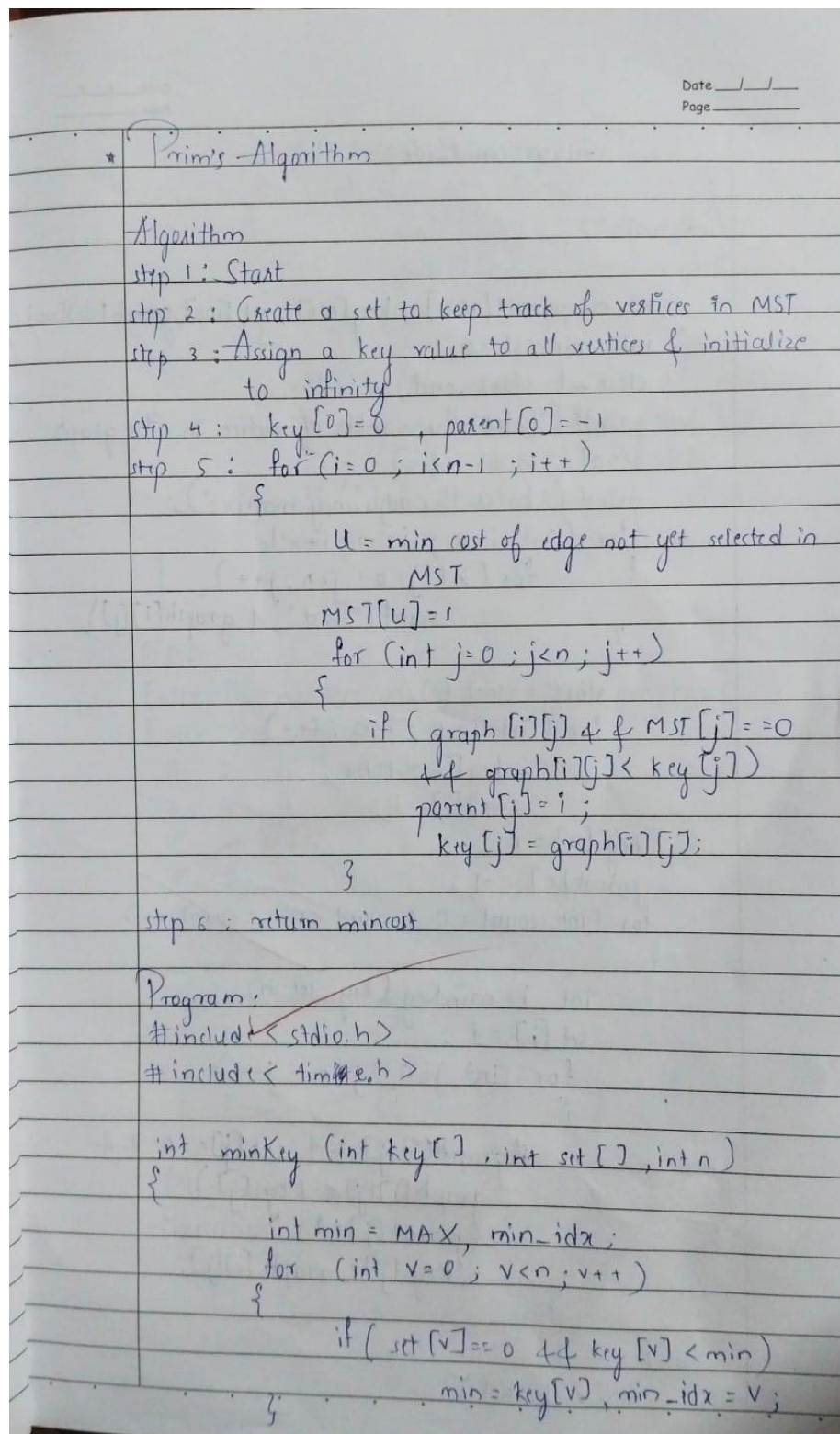
~~Does not work~~

Shortest distance matrix :

0	4	5	5	2
3	0	1	4	6
2	6	0	3	5
3	2	1	0	2
1	5	5	4	0

Execution time : 0.000000.

Prim's algorithm:



```

        return mid_idx;
    }

    int main()
    {
        int n, parent[100], key[100], set[100], graph[100][100];
        int mincost = 0;
        clock = t_start, end;
        printf("Enter the number of vertices in the graph:");
        scanf("%d", &n);
        printf("Enter the adjacency matrix:");
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
                scanf("%d", &graph[i][j]);
        }

        start = clock();
        for (int i = 0; i < n; i++)
            key[i] = MAX;
        set[i] = 0;
        key[0] = 0;
        parent[0] = -1;
        for (int count = 0; count < n-1; count++)
        {
            int i = minKey(key, set, n);
            set[i] = 1;
            for (int j = 0; j < n; j++)
            {
                if (graph[i][j] && set[j] == 0 &&
                    graph[i][j] < key[j])
                {
                    parent[j] = i;
                    key[j] = graph[i][j];
                }
            }
        }
        end = clock();
    }

```

```

printf(" Edge It Weight\n");
for (int i=1; i<n; i++)
{
    printf("%d-%d\t\t %d\n",
           parent[i], i, graph[parent[i]][i]);
    mincost += graph[parent[i]][i];
}

printf("\n Minimum cost is %d", mincost);
double time = (double)(end-start) /
               CLOCKS_PER_SEC;
printf("Execution time : %f", time);
return 0;
}

```

o/p:

Enter the number of vertices of the graph : 5

Enter the adjacency matrix :

0	2	0	6	0
2	0	3	0	5
0	3	0	0	7
6	0	0	0	9
0	5	7	9	0

Edge	Weight
0-1	2
1-2	3
0-3	6
1-4	5

Minimum cost : 16

Execution time : 0.00000

Kruskal's Algorithm:

* Kruskal's Algorithm

Step 1: Start

step 2: Initialize each vertex as a set

step 3: Initialize $i=0$, $mincost=0$

step 4: while ($i < n-1$ & edges left to be considered

{ find minCost of edge (u,v)

$j = \text{find}(u)$

$k = \text{find}(v)$

if ($j \neq k$)

{

$i++$;

$minCost += \text{cost}[u,v]$;

$t[i,1] = u$;

$t[i,2] = v$;

$\text{union}(j,k)$;

}

}

step 5: if ($i \neq n-1$)

return "No MST"

else

return MST

Step 6: Stop.

* Program

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
typedef struct
```

```
{ int u,v,w;
```

```
} Edge;
```

```
Edge edges[100], result[100];  
int parent[100];
```

```
int find(int i)  
{  
    while (parent[i] != i)  
        i = parent[i];  
    return i;  
}
```

```
void unionSet(int i, int j)  
{  
    parent[i] = j;  
}
```

```
void sortEdge(int e)  
{
```

```
    int i, j;
```

```
    Edge temp;
```

```
    for (i = 0; i < e - 1; i++)
```

```
    {
```

```
        for (j = 0; j < e - i - 1; j++)
```

```
        {  
            if (edges[j].w > edges[j+1].w)
```

```
            {  
                temp = edges[j];
```

```
                edges[j] = edges[j+1];
```

```
                edges[j+1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
int kruskals(int n, int e)
```

```
{  
    int count = 0;
```

```
    int cost = 0;
```

```
    int u, v;
```

```

for (int i=0; i<n; i++)
    parent[i]=i;
costEdges(e);
for (int i=0; i<e && count<n-1; i++)
{
    u = find(edges[i].u);
    v = find(edges[i].v);
    if (u==v)
    {
        result[count++] = edges[i];
        cost += edges[i].w;
        unionSet(u, v);
    }
}

printf("Minimum Spanning Tree: \n");
for (int i=0; i<count; i++)
{
    printf("%d -- %d == %d \n",
           result[i].u, result[i].v, result[i].w);
}

return cost;

```

```

int main()
{
    int n, e;
    printf("Enter the number of vertices:");
    scanf("%d", &n);
    printf("Enter the number of edges:");
    scanf("%d", &e);
    printf("Enter the edges (u, v, weight): \n");
    for (int i=0; i<e; i++)
    {
        scanf("%d %d %d", &edges[i].u,
              &edges[i].v, &edges[i].w);
    }
}

```



```
int cost = kruskals(n, e);
printf("Total cost = %d\n", cost);
return 0;
```

}

O/p:

Enter the numbers of vertices: 7

Enter the numbers of edges: 9

Enter the edges (u, v, weight):

1 2 4

1 3 28

2 5 26

3 6 12

3 4 8

4 5 22

4 7 16

6 7 6

5 7 20

Minimum Spanning Tree:

1 — 2 = 4

6 — 7 = 6

3 — 4 = 8

3 — 6 = 12

5 — 7 = 20

2 — 5 = 26

Total cost = 76.

~~Exhaustive~~

Fractional Knapsack:

* Knapsack problem using Greedy method

Algorithm:

- 1) Start
- 2) Read n & profit & weight of all ' n ' objects.
- 3) Sort them in decreasing order of $\frac{P[i]}{W[i]}$

```
4) for i=1 to n { x[i] = 0;
    remaining-capacity = m;
    for i=1 to n do
    {
        if (w[i] > remaining-capacity)
            break;
        remaining-capacity -= w[i]
    }
    if (i ≤ n)
        then x[i] =  $\frac{\text{remaining-capacity}}{w[i]}$ 
```

```
int profit = 0;
for i=1 to n
    profit += P[i] * x[i]
return profit;
```

Program:

```
#include <stdio.h>
```

```
int i, j, p[50], w[50], n, m;
float x[50];
```

```
int main()
```

```
{
    printf("Enter the number of objects:");
```

```
scanf("%d", &n);
```

```
printf("Enter the weights of all objects:");
```

```

for (i=0; i<n; i++)
    scanf ("%d", &w[i]);
printf ("Enter the profit of all objects:");
for (i=0; i<n; i++)
    scanf ("%d", &p[i]);
for (i=0; i<n-1; i++)
{
    for (j=i+1; j<n; j++)
    {
        if ((float)p[i]/w[i] < (float)
            p[j]/w[j])
        {
            int t1 = p[i];
            p[i] = p[j];
            p[j] = t1;
            int t2 = w[i];
            w[i] = w[j];
            w[j] = t2;
        }
    }
}

```

20
13/4

```

printf ("Enter the maximum capacity:");
scanf ("%d", &m);
for (i=0; i<n; i++)
{
    x[i] = 0;
}
int rem-cap = m;
for (i=0; i<n; i++)
{
    if (w[i] > rem-cap)
        break;
}

```



```

        x[i] = 1;
        rem-cap -= w[i]
    }
    if (i < n)
    {
        x[i] = (float) (rem-cap) / w[i];
    }
    float sum = 0;
    for (i = 0; i < n; i++)
    {
        sum = sum + (p[i] * x[i]);
    }
    printf("\n Total profit : %.1f", sum);
}

```

O/p:

Enter the number of objects : 3

Enter the weights of all objects : 18

15

0

Enter the profits associated with the weights : 25

24

15

Enter the maximum capacity : 20

Total profit : 31.50000.

Time-complexity:

$$= \sum_{i=0}^{n-1} \sum_{j=1}^{n-1} 1$$

$$= \sum_{i=0}^{n-2} (n-1-i)$$

$$= (n-1) \sum_{i=0}^{n-2} 1 = (n-1) (n-2+0+1)$$

$$= (n-1) (n-1) = (n-1)^2 \approx n^2$$

$$\therefore T(n) = O(n^2)$$

Dijkstra's Algorithm:

Date ___/___/___
Page _____

* Dijkstra's - Algorithm

```
#include <stdio.h>
#include <time.h>

// define n
// define in

int minDist (int dist[], int visited[])
{
    int min = 99999, min-index;
    for (int i = 0; i < n; i++)
    {
        if (visited[i] == 0 && dist[i] <= min)
        {
            min = dist[i];
            min-index = i;
        }
    }
    return min-index;
}

int dijkstra (int weight[n][n], int src)
{
    int dist[n];
    int visited[n];
    for (int i = 0; i < n; i++)
    {
        dist[i] = 99999;
        visited[i] = 0;
    }
    dist[src] = 0;
    for (int count = 0; count < n - 1; count++)
    {
        int u = minDist (dist, visited);
        visited[u] = 1;
    }
}
```

```

for (int v=0; v<n; v++)
{
    if (!visited[v] && weight[u][v] != 0)
        dist[v] = 99999;
    new-dist = dist[u] + weight[u][v];
    if (new-dist < dist[v])
        dist[v] = new-dist;
}
}

printf("Vertex It Distance from source\n");
for (int i=0; i<n; i++)
    printf("%d It It %d\n", i, dist[i]);
return 0;
}

```

```

int main()
{
    int i, j, weight[n][n];
    clock_t start, end;
    printf("Enter the weight matrix: (n^2)");
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            scanf("%d", &graph[i][j]);

    int source = 0;
    start = clock();
    dijkstra(graph, source);
    end = clock();

    double time_taken = (double)(end-start)
        / CLOCKS_PER_SEC;
    printf("Time taken is %d second",
        time_taken);
    return 0;
}

```


O/p:

Enter the weight matrix:

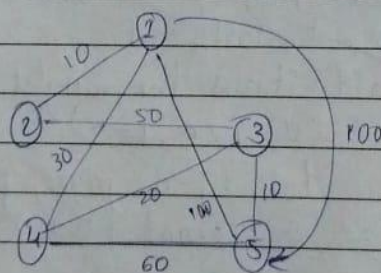
0	10	0	30	100
10	0	50	0	0
0	50	0	20	10
30	0	20	0	60
100	0	10	60	0

Vertex Distance from Source

0	0
2	10
3	50
4	30
5	60

Time taken: 0 seconds.

Graph:



Algorithm

- 1) Start
- 2) Initialize dist[n] with infinity (i.e. 99999) from 0 to n
- 3) Initialize visited[n] with 0 from 0 to n
- 4) set dist[source] = 0.
- 5) for i to n-1 do
 - u = min(dist, visited)
 - mark visited[u] = 1;

```

for v=0 to n do
  if (visited[v]==0 && weight[u][v]!=0
    && dist[u]!=aaaaa)
    new-dist = dist[u] + weight[u][v];
    if (new-dist < dist[v])
      dist[v] = new-dist
return dist[]

```

65 Stop.

Operation.	10	50	30		
	1	2	3	4	5
1	0	10	∞	30	100
2			60	30	100
4			50		90
3					60

Handwritten signature

N-Queens Problem:

Date ___/___/___
Page _____

* N-Queens Problem

- 1) Start
- 2) Place the queen at first cell of 1st row & 1st column
- 3) place Move to next row, place the queen in first cell. if;
→ it is in same column as previous queens
→ it move to next column. Check till the end; if get move to next column (or) move to previous row move the queen to next cell
- 4) Repeat '3' till the all the rows are filled.
- 5) Stop.

* Program

```
#include <stdio.h>
#include <stdbool.h>

int solutionCount = 0;

bool isSafe(int N, int board[N][N], int row, int col)
{
    int i, j;
    for (i = 0; i < col; i++)
        if (board[row][i])
            return false;
    for (i = row; j = col + 1; i >= 0 && j >= 0; i--, j++)
        if (board[i][j])
            return false;
    for (i = row; j = col - 1; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return false;
    return true;
}
```



```

        return false;

    for (i=row; i<N; i++)
        for (j=col; j>=0; j--)
            if (board[i][j])
                return false;

    return true;
}

void printSolution (int N, int board[N][N]) {
    printf("Solution: %d\n", ++solutionCount);
    for (int i=0; i<N; i++)
        for (int j=0; j<N; j++)
            printf("%d", board[i][j]);
}

void solveNQueen (int N, int board[N][N], int col) {
    if (col >= N)
        printSolution (N, board);
        return;
    for (int i=0; i<N; i++)
        if (isSafe (N, board, i, col)) {
            board[i][col] = 1;
            solveNQueen (N, board, col+1);
            board[i][col] = 0;
        }
}

void solveNQ (int N) {
    int board[N][N];
}

```

```

for (int i=0; i<N; i++)
    for (int j=0; j<N; j++)
        board[i][j] = 0;

solveNQueen(N, board, 0);
if (solutionCount == 0)
    printf("Solution does not exist\n");
else
    printf("Total solutions = %d\n",
        solutionCount);
}

int main()
{
    int N;
    printf("Enter value of N:");
    scanf("%d", &N);

    if (N <= 0)
    {
        printf("Enter a positive number: \n");
        return 0;
    }

    solveNQueen(N);
    return 0;
}

```

o/p: Enter value of N: 4

Solution 1: 0 0 1 0
 1 0 0 0
 0 0 0 1
 0 1 0 0

Solution 2 :

0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0

Total solutions = 2

[Signature]